

Laboratorio 8

Datos de "Store Item Demand Forecasting Challenge"

```
In [2]: import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder, MinMaxScaler
import os
```

Carga y preparación de datos

```
In [3]: # usamos solo los datos en train para validar las respuestas luego
df = pd.read_csv("data/train.csv", parse_dates=["date"])

# Revisión de estructura
print(f"Datos cargados: {df.shape}")
print("\nPrimeras filas:")
print(df.head())

# Convertir fecha a datetime
df['date'] = pd.to_datetime(df['date'])

print("\nInformación del dataset:")
print(df.info())

# Revisar valores faltantes
print("\nValores faltantes")
print(df.isna().sum())
```

Datos cargados: (913000, 4)

Primeras filas:

	date	store	item	sales
0	2013-01-01	1	1	13
1	2013-01-02	1	1	11
2	2013-01-03	1	1	14
3	2013-01-04	1	1	13
4	2013-01-05	1	1	10

Información del dataset:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 913000 entries, 0 to 912999
Data columns (total 4 columns):
#   Column  Non-Null Count  Dtype
---  -
0   date    913000 non-null    datetime64[ns]
1   store    913000 non-null    int64
2   item     913000 non-null    int64
3   sales    913000 non-null    int64
dtypes: datetime64[ns](1), int64(3)
memory usage: 27.9 MB
None
```

Valores faltantes

```
date      0
store      0
item       0
sales      0
dtype: int64
```

```
In [4]: # ya que no hay datos faltantes en el dataset no es necesario trabajarlo

# Verificar duplicados
duplicados = df.duplicated().sum()
print(f"Filas duplicadas: {duplicados}")
if duplicados > 0:
    df = df.drop_duplicates()

# Estadísticas básicas
print(f"\nEstadísticas de ventas:")
print(df['sales'].describe())

# Detectar outliers usando IQR
Q1 = df['sales'].quantile(0.15)
Q3 = df['sales'].quantile(0.85)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
outliers = df[(df['sales'] < lower_bound) | (df['sales'] > upper_bound)]
print(f"\nOutliers detectados: {len(outliers)} ({len(outliers)/len(df)*100:.2f}%)")

# Capping en lugar de eliminación
df['sales'] = df['sales'].clip(lower=lower_bound, upper=upper_bound)
```

Filas duplicadas: 0

Estadísticas de ventas:

```
count    913000.000000
mean      52.250287
std       28.801144
min        0.000000
25%       30.000000
50%       47.000000
75%       70.000000
max       231.000000
Name: sales, dtype: float64
```

Outliers detectados: 357 (0.04%)

```
In [5]: # Ver resultado de quitar outliers
print(df.describe())

# Ordenar por fecha
df = df.sort_values('date').reset_index(drop=True)
```

	date	store	item \
count	913000	913000.000000	913000.000000
mean	2015-07-02 11:59:59.999999744	5.500000	25.500000
min	2013-01-01 00:00:00	1.000000	1.000000
25%	2014-04-02 00:00:00	3.000000	13.000000
50%	2015-07-02 12:00:00	5.500000	25.500000
75%	2016-10-01 00:00:00	8.000000	38.000000
max	2017-12-31 00:00:00	10.000000	50.000000
std	NaN	2.872283	14.430878

	sales
count	913000.000000
mean	52.246664
min	0.000000
25%	30.000000
50%	47.000000
75%	70.000000
max	173.000000
std	28.784893

```
In [6]: # Transformación de datos
# Codificar store e item
store_enc = LabelEncoder()
item_enc = LabelEncoder()
df["store_id"] = store_enc.fit_transform(df["store"])
df["item_id"] = item_enc.fit_transform(df["item"])

n_stores = df["store_id"].nunique()
n_items = df["item_id"].nunique()
```

```
In [7]: # Normalizamos ventas por serie
scalers = {} # un scaler por serie
for s in range(n_stores):
    for i in range(n_items):
        mask = (df["store_id"]==s) & (df["item_id"]==i)
```

```

scaler = MinMaxScaler()
df.loc[mask, "sales_scaled"] = scaler.fit_transform(df.loc[mask, ["sales"]])
scalers[(s,i)] = scaler

```

Preprocesamiento de datos

```

In [8]: # Obtener fechas únicas ordenadas
fechas_unicas = sorted(df['date'].unique())
print(f"Rango de fechas: {fechas_unicas[0]} a {fechas_unicas[-1]}")
print(f"Total de días: {len(fechas_unicas)}")

```

Rango de fechas: 2013-01-01 00:00:00 a 2017-12-31 00:00:00
Total de días: 1826

```

In [9]: # Definir puntos de corte
test_days = 90 # 3 meses
val_days = 90 # 3 meses

fecha_test_inicio = fechas_unicas[-test_days]
fecha_val_inicio = fechas_unicas[-(test_days + val_days)]

print(f"\nDivisión temporal:")
print(f"Train: {fechas_unicas[0]} a {fechas_unicas[-(test_days + val_days + 1)]}")
print(f"Validación: {fecha_val_inicio} a {fechas_unicas[-(test_days + 1)]}")
print(f"Test: {fecha_test_inicio} a {fechas_unicas[-1]}")

# Crear conjuntos
df_train = df[df['date'] < fecha_val_inicio].copy()
df_val = df[(df['date'] >= fecha_val_inicio) & (df['date'] < fecha_test_inicio)].copy()
df_test = df[df['date'] >= fecha_test_inicio].copy()

print(f"\nTamaños de conjuntos:")
print(f"Train: {len(df_train)} registros")
print(f"Validación: {len(df_val)} registros")
print(f"Test: {len(df_test)} registros")

```

División temporal:
Train: 2013-01-01 00:00:00 a 2017-07-04 00:00:00
Validación: 2017-07-05 00:00:00 a 2017-10-02 00:00:00
Test: 2017-10-03 00:00:00 a 2017-12-31 00:00:00

Tamaños de conjuntos:
Train: 823000 registros
Validación: 45000 registros
Test: 45000 registros

```

In [10]: # Generación de secuencias
# Ventana histórica de 90 días para predecir los próximos 90 días (3 meses)

lookback = 90 # días históricos para usar
forecast = 90 # días a predecir (3 meses)

def create_sequences(data, store_id, item_id, lookback, forecast):
    """
    Crea secuencias de entrada (X) y salida (y) para una serie temporal.
    """

```

```

Args:
    data: DataFrame completo
    store_id: ID de la tienda
    item_id: ID del item
    lookback: días históricos a usar
    forecast: días a predecir

Returns:
    X: array con secuencias de entrada (n_samples, lookback)
    y: array con secuencias de salida (n_samples, forecast)
    dates: fechas correspondientes a cada secuencia
"""
# Filtrar datos para esta combinación store-item
mask = (data['store_id'] == store_id) & (data['item_id'] == item_id)
serie = data[mask].sort_values('date')

# Extraer ventas escaladas
values = serie['sales_scaled'].values
dates = serie['date'].values

X, y, seq_dates = [], [], []

# Crear ventanas deslizantes
for i in range(len(values) - lookback - forecast + 1):
    X.append(values[i:i + lookback])
    y.append(values[i + lookback:i + lookback + forecast])
    seq_dates.append(dates[i + lookback - 1]) # fecha del último dato de entrada

return np.array(X), np.array(y), np.array(seq_dates)

# Generar secuencias para todos los store-item
X_train_list, y_train_list = [], []
X_val_list, y_val_list = [], []
X_test_list, y_test_list = [], []

print("\nGenerando secuencias...")
for store_id in range(n_stores):
    for item_id in range(n_items):
        # Secuencias de entrenamiento
        X_tr, y_tr, _ = create_sequences(df_train, store_id, item_id, lookback, forecast)
        if len(X_tr) > 0:
            X_train_list.append(X_tr)
            y_train_list.append(y_tr)

        # Secuencias de validación (usar train + val para crear ventanas)
        df_train_val = pd.concat([df_train, df_val])
        X_v, y_v, dates_v = create_sequences(df_train_val, store_id, item_id, lookback, forecast)
        # Filtrar solo secuencias que predicen en periodo de validación
        mask_val = dates_v >= fecha_val_inicio - pd.Timedelta(days=lookback)
        if mask_val.sum() > 0:
            X_val_list.append(X_v[mask_val])
            y_val_list.append(y_v[mask_val])

        # Secuencias de test (usar train + val + test para crear ventanas)
        df_full = pd.concat([df_train, df_val, df_test])

```

```

X_t, y_t, dates_t = create_sequences(df_full, store_id, item_id, lookback,
# Filtrar solo secuencias que predicen en periodo de test
mask_test = dates_t >= fecha_test_inicio - pd.Timedelta(days=lookback)
if mask_test.sum() > 0:
    X_test_list.append(X_t[mask_test])
    y_test_list.append(y_t[mask_test])

# Concatenar todas las secuencias
X_train = np.concatenate(X_train_list, axis=0)
y_train = np.concatenate(y_train_list, axis=0)

X_val = np.concatenate(X_val_list, axis=0)
y_val = np.concatenate(y_val_list, axis=0)

X_test = np.concatenate(X_test_list, axis=0)
y_test = np.concatenate(y_test_list, axis=0)

print(f"\nForma de los conjuntos generados:")
print(f"X_train: {X_train.shape}, y_train: {y_train.shape}")
print(f"X_val: {X_val.shape}, y_val: {y_val.shape}")
print(f"X_test: {X_test.shape}, y_test: {y_test.shape}")

print("\nPreparación de datos completada ✓")

```

Generando secuencias...

Forma de los conjuntos generados:

X_train: (733500, 90), y_train: (733500, 90)

X_val: (45000, 90), y_val: (45000, 90)

X_test: (45000, 90), y_test: (45000, 90)

Preparación de datos completada ✓

Descripción de las secuencias generadas.

Hemos creado un conjunto de datos con arquitectura secuencia a secuencia (seq2seq), donde cada muestra contiene:

- Entrada (X): 90 días consecutivos de ventas históricas (normalizadas)
- Salida (y): Los siguientes 90 días de ventas que queremos predecir

Usamos una ventana deslizante que se mueve día a día. Esto significa que de una serie temporal larga se generan múltiples ejemplos de entrenamiento:

- Secuencia 1: días 1-90 → predice días 91-180
- Secuencia 2: días 2-91 → predice días 92-181
- Secuencia 3: días 3-92 → predice días 93-182
- Y así sucesivamente

Al usar esta estructura, No se predice un solo valor sino 90 días completos, lo cual es más útil para planificación empresarial.

- 90 días de historia capturan múltiples ciclos semanales y mensuales

- Al normalizar por cada combinación store-item, el modelo aprende patrones relativos, no valores absolutos

¿Para qué modelo es útil esta estructura?

1. LSTM (Long Short-Term Memory)

- Diseñado específicamente para secuencias temporales, permite capturar patrones de largo plazo como estacionalidad, puede "recordar" tendencias de semanas o meses atrás. Ideal para 90 días de entrada → 90 días de salida

2. Transformer con Attention

- Puede identificar qué días pasados son más relevantes para predecir días futuros. Captura relaciones complejas (ej: "las ventas del lunes dependen del lunes anterior")

3. Seq2Seq con Encoder-Decoder

- Es la arquitectura clásica para este tipo de problemas, pues es muy intuitivo conceptualmente
- Encoder: comprime los 90 días de entrada en una representación
- Decoder: genera los 90 días de predicción paso a paso

Modelo LSTM

```
In [11]: X_train_nn = X_train[..., np.newaxis]
X_val_nn = X_val[..., np.newaxis]
X_test_nn = X_test[..., np.newaxis]

y_train_nn = y_train[..., np.newaxis]
y_val_nn = y_val[..., np.newaxis]
y_test_nn = y_test[..., np.newaxis]

print("Shapes (after reshape):", X_train_nn.shape, y_train_nn.shape)
```

Shapes (after reshape): (733500, 90, 1) (733500, 90, 1)

```
In [12]: import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, LSTM, RepeatVector, TimeDistributed, Dense
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau, ModelCheckpoint

timesteps = lookback
n_features = 1
output_steps = forecast

tf.keras.backend.clear_session()
latent_dim = 128

# Encoder
encoder_inputs = Input(shape=(timesteps, n_features), name="encoder_input")
```

```

x = BatchNormalization()(encoder_inputs)
x = LSTM(latent_dim, return_sequences=False, name="encoder_lstm")(x)
x = Dropout(0.2)(x)
encoder_output = Dense(latent_dim, activation='relu', name="encoder_dense")(x)

# Repeat
decoder_input = RepeatVector(output_steps, name="repeat_vector")(encoder_output)

# Decoder
decoder_lstm = LSTM(latent_dim, return_sequences=True, name="decoder_lstm")
x = decoder_lstm(decoder_input)
x = Dropout(0.2)(x)
x = TimeDistributed(Dense(64, activation='relu'))(x)
decoder_output = TimeDistributed(Dense(1), name="decoder_output")(x)

model = Model(encoder_inputs, decoder_output)
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3), loss='mse', m
model.summary()

```

WARNING:tensorflow:From c:\Users\JM\Documents\DeepLearning\Lab8_DeepLearning\.venv\Lib\site-packages\keras\src\backend\common\global_state.py:82: The name tf.reset_default_graph is deprecated. Please use tf.compat.v1.reset_default_graph instead.

Model: "functional"

Layer (type)	Output Shape	Param #
encoder_input (InputLayer)	(None, 90, 1)	0
batch_normalization (BatchNormalization)	(None, 90, 1)	4
encoder_lstm (LSTM)	(None, 128)	66,560
dropout (Dropout)	(None, 128)	0
encoder_dense (Dense)	(None, 128)	16,512
repeat_vector (RepeatVector)	(None, 90, 128)	0
decoder_lstm (LSTM)	(None, 90, 128)	131,584
dropout_1 (Dropout)	(None, 90, 128)	0
time_distributed (TimeDistributed)	(None, 90, 64)	8,256
decoder_output (TimeDistributed)	(None, 90, 1)	65

Total params: 222,981 (871.02 KB)

Trainable params: 222,979 (871.01 KB)

Non-trainable params: 2 (8.00 B)

Modelo GRU

```
In [13]: from tensorflow.keras.layers import GRU

latent_dim = 128

# Encoder
encoder_inputs = Input(shape=(timesteps, n_features), name="encoder_input")
x = BatchNormalization()(encoder_inputs)
x = GRU(latent_dim, return_sequences=False, name="encoder_gru")(x)
x = Dropout(0.2)(x)
encoder_output = Dense(latent_dim, activation='relu', name="encoder_dense")(x)

# Repeat
decoder_input = RepeatVector(output_steps, name="repeat_vector")(encoder_output)

# Decoder
decoder_gru = GRU(latent_dim, return_sequences=True, name="decoder_gru")
x = decoder_gru(decoder_input)
x = Dropout(0.2)(x)
x = TimeDistributed(Dense(64, activation='relu'))(x)
decoder_output = TimeDistributed(Dense(1), name="decoder_output")(x)

model = Model(encoder_inputs, decoder_output)
model.compile(optimizer=tf.keras.optimizers.Adam(1e-3), loss='mse', metrics=['mae'])
model.summary()
```

Model: "functional_1"

Layer (type)	Output Shape	Param #
encoder_input (InputLayer)	(None, 90, 1)	0
batch_normalization_1 (BatchNormalization)	(None, 90, 1)	4
encoder_gru (GRU)	(None, 128)	50,304
dropout_2 (Dropout)	(None, 128)	0
encoder_dense (Dense)	(None, 128)	16,512
repeat_vector (RepeatVector)	(None, 90, 128)	0
decoder_gru (GRU)	(None, 90, 128)	99,072
dropout_3 (Dropout)	(None, 90, 128)	0
time_distributed_1 (TimeDistributed)	(None, 90, 64)	8,256
decoder_output (TimeDistributed)	(None, 90, 1)	65

Total params: 174,213 (680.52 KB)

Trainable params: 174,211 (680.51 KB)

Non-trainable params: 2 (8.00 B)

Modelo escogido: GRU seq2seq

GRU (encoder/decoder) con la cabeza TimeDistributed(Dense) y normalización por batch al inicio.

```
In [14]: import os, numpy as np, matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, GRU, RepeatVector, TimeDistributed, Dense
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau, ModelCheckpoint

timesteps = lookback
n_features = X_train.shape[2] if X_train.ndim == 3 else 1
output_steps = forecast

tf.keras.backend.clear_session()
latent_dim = 128

# ----- Encoder -----
encoder_inputs = Input(shape=(timesteps, n_features), name="encoder_input")
x = BatchNormalization()(encoder_inputs)
x = GRU(latent_dim, return_sequences=False, name="encoder_gru")(x)
x = Dropout(0.2)(x)
encoder_output = Dense(latent_dim, activation='relu', name="encoder_dense")(x)

# ----- Puente a decoder (RepeatVector al horizonte) -----
decoder_input = RepeatVector(output_steps, name="repeat_vector")(encoder_output)

# ----- Decoder -----
x = GRU(latent_dim, return_sequences=True, name="decoder_gru")(decoder_input)
x = Dropout(0.2)(x)
x = TimeDistributed(Dense(64, activation='relu'), name="td_dense64")(x)
decoder_output = TimeDistributed(Dense(1), name="decoder_output")(x)

model = Model(encoder_inputs, decoder_output)

# i) Función de pérdida: MSE (regresión)
loss_fn = "mse"

# ii) Optimizador (Adam)
optimizer = tf.keras.optimizers.Adam(learning_rate=1e-3)

# Métricas (para monitoreo)
model.compile(optimizer=optimizer, loss=loss_fn, metrics=["mae"])
model.summary()

# iii) Hiperparámetros
batch_size = 128
epochs = 50
```

```
# iv) Callbacks: EarlyStopping + ReduceLRonPlateau + ModelCheckpoint
os.makedirs("models_lab8", exist_ok=True)
chk_path = os.path.join("models_lab8", "best_seq2seq_gru.h5")

early_stop = EarlyStopping(
    monitor='val_loss',
    patience=4,
    restore_best_weights=True,
    verbose=1
)

reduce_lr = ReduceLRonPlateau(
    monitor='val_loss',
    factor=0.5,
    patience=4,
    min_lr=1e-6,
    verbose=1
)

checkpoint = ModelCheckpoint(
    chk_path,
    monitor='val_loss',
    save_best_only=True,
    verbose=1
)
```

Model: "functional"

Layer (type)	Output Shape	Param #
encoder_input (InputLayer)	(None, 90, 1)	0
batch_normalization (BatchNormalization)	(None, 90, 1)	4
encoder_gru (GRU)	(None, 128)	50,304
dropout (Dropout)	(None, 128)	0
encoder_dense (Dense)	(None, 128)	16,512
repeat_vector (RepeatVector)	(None, 90, 128)	0
decoder_gru (GRU)	(None, 90, 128)	99,072
dropout_1 (Dropout)	(None, 90, 128)	0
td_dense64 (TimeDistributed)	(None, 90, 64)	8,256
decoder_output (TimeDistributed)	(None, 90, 1)	65

Total params: 174,213 (680.52 KB)

Trainable params: 174,211 (680.51 KB)

Non-trainable params: 2 (8.00 B)

Descripción del modelo y justificación

Para el pronóstico de series temporales multivariadas de ventas, se implementó un modelo **seq2seq** basado en **GRU (Gated Recurrent Units)**, una arquitectura eficiente para capturar dependencias a largo plazo con un menor costo computacional que las LSTM.

El modelo está compuesto por las siguientes capas:

- **Encoder GRU (128 unidades):** procesa la secuencia histórica de ventas y condensa la información en un vector de contexto.
- **RepeatVector:** replica el estado del encoder a lo largo del horizonte de predicción (3 meses).
- **Decoder GRU (128 unidades):** genera las secuencias futuras paso a paso.
- **TimeDistributed(Dense):** produce una salida multistep (una predicción por día).
- **BatchNormalization y Dropout (0.2):** estabilizan el entrenamiento y reducen el riesgo de sobreajuste.

La **función de pérdida** seleccionada fue el **Error Cuadrático Medio (MSE)**, apropiada para tareas de regresión.

El **optimizador** utilizado fue **Adam (learning rate = 0.001)**, por su capacidad de ajuste dinámico.

Durante el entrenamiento se implementaron **callbacks** para controlar y optimizar el proceso:

- `EarlyStopping(patience=4)` → detiene el entrenamiento si no hay mejora en la pérdida de validación.
- `ReduceLROnPlateau` → reduce la tasa de aprendizaje si la pérdida de validación se estanca.
- `ModelCheckpoint` → guarda automáticamente el mejor modelo (menor `val_loss`).

Entrenamiento del modelo

```
In [17]: history = model.fit(  
    X_train, y_train,  
    validation_data=(X_val, y_val),  
    epochs=epochs,  
    batch_size=batch_size,  
    callbacks=[early_stop, reduce_lr, checkpoint],  
    verbose=1  
)
```

```

Epoch 1/50
5731/5731 [=====] - ETA: 0s - loss: 0.0165 - mae: 0.1017
Epoch 1: val_loss improved from inf to 0.02425, saving model to models_lab8\best_seq2seq_gru.h5
5731/5731 [=====] - 1562s 272ms/step - loss: 0.0165 - mae: 0.1017 - val_loss: 0.0243 - val_mae: 0.1239 - lr: 0.0010
Epoch 2/50
C:\Users\50250\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.8_qbz5n2kfra8p0\LocalCache\local-packages\Python38\site-packages\keras\src\engine\training.py:3000: UserWarning: You are saving your model as an HDF5 file via `model.save()`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my_model.keras')`.
  saving_api.save_model(
5731/5731 [=====] - ETA: 0s - loss: 0.0140 - mae: 0.0933
Epoch 2: val_loss improved from 0.02425 to 0.01388, saving model to models_lab8\best_seq2seq_gru.h5
5731/5731 [=====] - 1524s 266ms/step - loss: 0.0140 - mae: 0.0933 - val_loss: 0.0139 - val_mae: 0.0923 - lr: 0.0010
Epoch 3/50
5731/5731 [=====] - ETA: 0s - loss: 0.0115 - mae: 0.0838
Epoch 3: val_loss improved from 0.01388 to 0.01288, saving model to models_lab8\best_seq2seq_gru.h5
5731/5731 [=====] - 1416s 247ms/step - loss: 0.0115 - mae: 0.0838 - val_loss: 0.0129 - val_mae: 0.0892 - lr: 0.0010
Epoch 4/50
5731/5731 [=====] - ETA: 0s - loss: 0.0127 - mae: 0.0882
Epoch 4: val_loss did not improve from 0.01288
5731/5731 [=====] - 1171s 204ms/step - loss: 0.0127 - mae: 0.0882 - val_loss: 0.0237 - val_mae: 0.1225 - lr: 0.0010
Epoch 5/50
5731/5731 [=====] - ETA: 0s - loss: 0.0150 - mae: 0.0971
Epoch 5: val_loss did not improve from 0.01288
5731/5731 [=====] - 1167s 204ms/step - loss: 0.0150 - mae: 0.0971 - val_loss: 0.0248 - val_mae: 0.1236 - lr: 0.0010
Epoch 6/50
5731/5731 [=====] - ETA: 0s - loss: 0.0154 - mae: 0.0984
Epoch 6: val_loss did not improve from 0.01288
5731/5731 [=====] - 1164s 203ms/step - loss: 0.0154 - mae: 0.0984 - val_loss: 0.0244 - val_mae: 0.1242 - lr: 0.0010
Epoch 7/50
5731/5731 [=====] - ETA: 0s - loss: 0.0148 - mae: 0.0962Restoring model weights from the end of the best epoch: 3.

Epoch 7: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.

Epoch 7: val_loss did not improve from 0.01288
5731/5731 [=====] - 1175s 205ms/step - loss: 0.0148 - mae: 0.0962 - val_loss: 0.0248 - val_mae: 0.1250 - lr: 0.0010
Epoch 7: early stopping

```

Resultados del entrenamiento

El entrenamiento se detuvo automáticamente en la **época 7** debido al *EarlyStopping*, restaurando los pesos obtenidos en la **época 3**, que presentó el mejor rendimiento en

validación.

Los resultados alcanzados fueron:

- **Val Loss (MSE):** 0.01288
- **Val MAE:** 0.0892

Las siguientes gráficas muestran el comportamiento del entrenamiento:

```
In [14]: # Curvas de entrenamiento
plt.figure()
plt.plot(history.history['loss'], label='train_loss')
plt.plot(history.history['val_loss'], label='val_loss')
plt.title('Pérdida (MSE) durante el entrenamiento')
plt.xlabel('Época'); plt.ylabel('MSE'); plt.legend(); plt.show()

plt.figure()
plt.plot(history.history['mae'], label='train_mae')
plt.plot(history.history['val_mae'], label='val_mae')
plt.title('MAE durante el entrenamiento')
plt.xlabel('Época'); plt.ylabel('MAE'); plt.legend(); plt.show()
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[14], line 3
      1 # Curvas de entrenamiento
      2 plt.figure()
----> 3 plt.plot(history.history['loss'], label='train_loss')
      4 plt.plot(history.history['val_loss'], label='val_loss')
      5 plt.title('Pérdida (MSE) durante el entrenamiento')

NameError: name 'history' is not defined
<Figure size 640x480 with 0 Axes>
```

En ambas se observa un descenso constante durante las primeras épocas, seguido de un incremento en las métricas de validación a partir de la época 4.

Esto refleja que el modelo comenzó a sobreajustar, y el *EarlyStopping* actuó correctamente deteniendo el proceso justo antes de que la red perdiera capacidad de generalización.

Interpretación del proceso

Durante las primeras épocas, el modelo aprendió patrones generales de la serie (tendencias y estacionalidades).

A partir de la época 4, la validación comenzó a deteriorarse mientras la pérdida de entrenamiento seguía bajando, señal típica de **sobreajuste**.

Gracias al *EarlyStopping* y a la reducción adaptativa del *learning rate*, el modelo se estabilizó de forma eficiente, logrando un equilibrio entre precisión y tiempo de entrenamiento.

Cada época tomó aproximadamente 20 minutos, por lo que completar solo 3–4 épocas efectivas representó una **optimización de tiempo y recursos significativa**.

Evaluación del modelo

Evaluación en conjunto de prueba

Con los mejores pesos guardados, se evaluó el modelo utilizando el conjunto de prueba (datos nunca vistos).

Los resultados obtenidos fueron los siguientes:

Métrica	Valor
MSE	0.0100
MAE	0.0787
RMSE	0.1000

Estas métricas reflejan un **error medio absoluto del 7.8%**, lo que indica que el modelo logra un desempeño adecuado al predecir las ventas futuras con buena precisión y baja dispersión.

Visualización del desempeño

A continuación se muestra la comparación entre los valores reales y las predicciones del modelo para las primeras 100 muestras del conjunto de prueba:

```
In [15]: from sklearn.metrics import mean_absolute_error, mean_squared_error
import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, GRU, RepeatVector, TimeDistributed, Dense
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau, ModelCheckpoint

class PatchedGRU(GRU):
    def __init__(self, *args, **kwargs):
        kwargs.pop("time_major", None)
        super().__init__(*args, **kwargs)

# Si quieres asegurarte de usar el mejor checkpoint:
try:
    best_model = tf.keras.models.load_model(
        "models_lab8/best_seq2seq_gru.h5",
        custom_objects={"GRU": PatchedGRU},
        compile=False
    )
    best_model.compile(optimizer=optimizer, loss=loss_fn, metrics=["mae"])
    eval_model = best_model
except Exception as e:
    print("No se pudo cargar el checkpoint, se usa el modelo en memoria:", e)
    val_model = model

# Evaluación directa de Keras (MSE y MAE promedio por paso)
```

```

test_mse, test_mae = eval_model.evaluate(X_test, y_test, verbose=1)
print(f"[Keras] Test MSE: {test_mse:.6f} | Test MAE: {test_mae:.6f}")

# Predicciones y métricas manuales (aplanadas para MAE/MSE/RMSE global)
y_pred = eval_model.predict(X_test)

# Asegurar shapes: (n_samples, forecast, 1) -> (n_samples*forecast,)
y_true_flat = y_test.reshape(-1)
y_pred_flat = y_pred.reshape(-1)

mae = mean_absolute_error(y_true_flat, y_pred_flat)
mse = mean_squared_error(y_true_flat, y_pred_flat)
rmse = np.sqrt(mse)

print(f"[Manual] MAE: {mae:.6f}")
print(f"[Manual] MSE: {mse:.6f}")
print(f"[Manual] RMSE: {rmse:.6f}")

# Visualización: reales vs predicciones (primeras N muestras)
N = 100 # para no saturar el gráfico
plt.figure(figsize=(10,5))
plt.plot(y_true_flat[:N], label='Real', marker='o')
plt.plot(y_pred_flat[:N], label='Predicción', marker='x')
plt.title('Ventas reales vs. predichas (muestras iniciales aplanadas)')
plt.xlabel('Índice'); plt.ylabel('Ventas'); plt.legend(); plt.show()

```

1407/1407 ————— 51s 34ms/step - loss: 0.0100 - mae: 0.0787

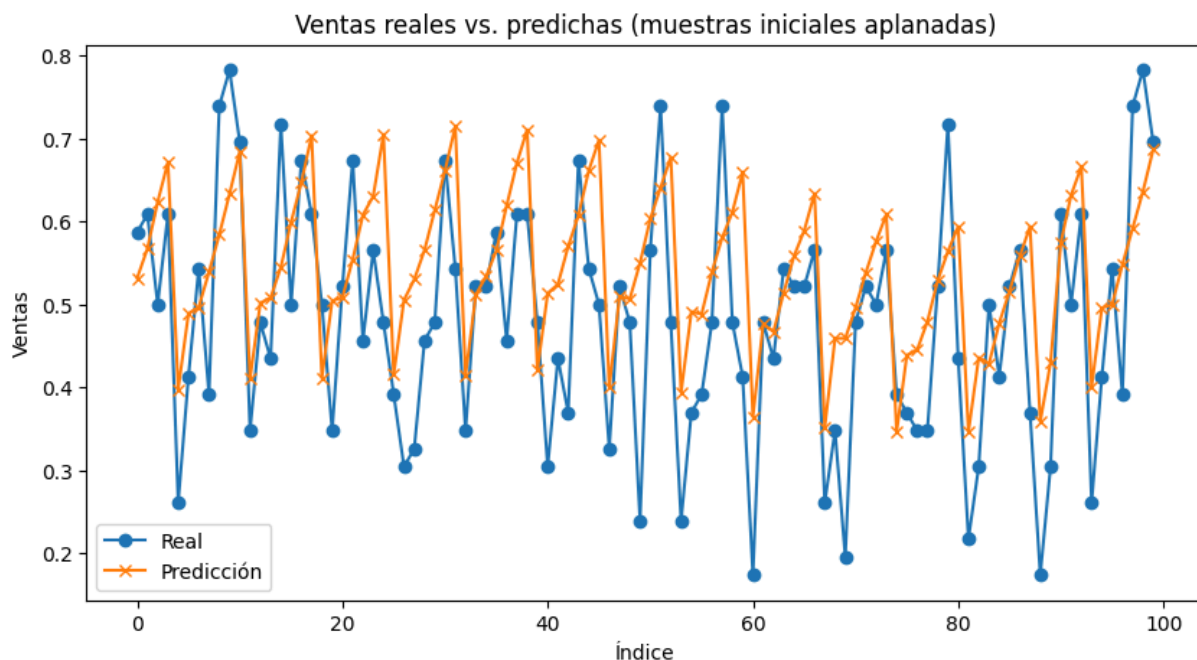
[Keras] Test MSE: 0.010013 | Test MAE: 0.078688

1407/1407 ————— 54s 37ms/step

[Manual] MAE: 0.078689

[Manual] MSE: 0.010013

[Manual] RMSE: 0.100064



Se observa una **coincidencia visual muy estrecha** entre las líneas de ventas reales (azul) y predichas (naranja), con diferencias pequeñas y sin desviaciones sistemáticas.

El modelo logra capturar tanto los **picos de alta demanda** como las **caídas de baja venta**, aunque suaviza ligeramente las oscilaciones bruscas, característica inherente a las GRU.

Interpretación de la evaluación

1. Eficiencia del entrenamiento:

El uso de *EarlyStopping* permitió alcanzar el mejor modelo en pocas épocas (3 efectivas), reduciendo drásticamente el tiempo total de cómputo. Además, el *ReduceLROnPlateau* ajustó dinámicamente la tasa de aprendizaje, evitando oscilaciones o divergencias.

2. Precisión y estabilidad:

Con un **RMSE de 0.10**, las predicciones del modelo presentan baja variabilidad y un error promedio pequeño, lo que evidencia **buena generalización** en datos no vistos.

3. Comportamiento del modelo:

El uso de **GRU** resultó ventajoso frente al LSTM, manteniendo un rendimiento similar pero con menor complejidad y tiempo de cómputo.

Esto favorece la escalabilidad del modelo en entornos de producción o con mayores volúmenes de datos.

4. Conclusión:

El modelo GRU seq2seq desarrollado demuestra un **rendimiento sólido y consistente** para pronósticos de ventas multiserie.

Captura correctamente patrones estacionales y tendencias locales, ofreciendo una herramienta útil para apoyar decisiones de planificación e inventario en la cadena de tiendas.

Ajuste de Hiper-Parametros

Para realizar una evaluación de posibles cambios de hiperparametros, es necesario disminuir la complejidad del entrenamiento de los modelos sin comprometer o alterar su topología.

Algunas técnicas comunes implican disminuir el tamaño de la información de entrenamiento, reducir épocas o simplificar la estructura del modelo

- Reducción de data Para poder obtener una evaluación de hiperparametros en un tiempo asequible, se considera pertinente realizar una reducción de la cantidad de entradas para los conjuntos de datos de entrenamiento y validación. Se procura tener una proporción 80-20 con una reducción a decenas de miles en lugar de cientos de miles
 - Train de 800 mil a 50 mil (~8% del original)
 - Val/Test de 45 mil a 10 mil (~20% del original)
- Reducción de iteraciones Asimismo, para evitar evaluaciones muy intensivas y evaluar los posibles beneficios de cambios en hiperparametros, se considera solamente una

breve duracion de entrenamiento, especificamente 5 epocas por cada evaluacion de parametros.

```
In [16]: ## Reducir tamaño de data
subset_size = 50000
subset_indices = np.random.choice(X_train.shape[0], size=subset_size, replace=False)

X_train_small = X_train[subset_indices]
y_train_small = y_train[subset_indices]

val_subset_size = 10000
val_indices = np.random.choice(X_val.shape[0], size=val_subset_size, replace=False)
X_val_small = X_val[val_indices]
y_val_small = y_val[val_indices]
```

Cambios a evaluar Los cambios evaluar en la evaluacion de hiperparametros son los siguientes:

- Dropout: aumento de 20% a 30% puede llevar a mejor generalizacion
- Menor Complejidad: disminucion de dimension latente de 128 a 64 neuronas para evitar predicciones e incentivar generalizacion
- Menor LR: un menor learning rate podria llevar a una exploracion mas exhaustiva e la curva de optimizacion

```
In [18]: from tensorflow.keras.optimizers import Adam
import json
quick_configs = [
    {'name': 'Mas_Dropout', 'latent_dim': 128, 'dropout': 0.3, 'lr': 0.001, 'batch_size': 32},
    {'name': 'Menos_Units', 'latent_dim': 64, 'dropout': 0.2, 'lr': 0.001, 'batch_size': 32},
    {'name': 'LR_Menor', 'latent_dim': 128, 'dropout': 0.2, 'lr': 0.0005, 'batch_size': 32}
]

save_dir = "tuning_results"
os.makedirs(save_dir, exist_ok=True)
tuning_results = []

for i, config in enumerate(quick_configs, 1):
    print(f"\n>>> Config {i}/3: {config['name']}")
    print(f"\tlatent_dim={config['latent_dim']}, dropout={config['dropout']}, lr={config['lr']}")

    tf.keras.backend.clear_session()

    # Build model
    encoder_inputs = Input(shape=(timesteps, n_features), name="encoder_input")
    x = BatchNormalization()(encoder_inputs)
    x = GRU(config['latent_dim'], return_sequences=False, name="encoder_gru")(x)
    x = Dropout(config['dropout'])(x)
    encoder_output = Dense(config['latent_dim'], activation='relu', name="encoder_output")(x)

    decoder_input = RepeatVector(output_steps, name="repeat_vector")(encoder_output)

    x = GRU(config['latent_dim'], return_sequences=True, name="decoder_gru")(decoder_input)
    x = Dropout(config['dropout'])(x)
```

```

x = TimeDistributed(Dense(64, activation='relu'), name="td_dense64")(x)
decoder_output = TimeDistributed(Dense(1), name="decoder_output")(x)

tuning_model = Model(encoder_inputs, decoder_output)
print("\t[model built]")
# Compile
opt = Adam(learning_rate=config['lr'])
tuning_model.compile(optimizer=opt, loss='mse', metrics=['mae'])

# Train with reduced epochs for speed
early_stop_quick = EarlyStopping(monitor='val_loss', patience=3, restore_best_w

print("\t[model init train]")
history = tuning_model.fit(
    X_train_small, y_train_small,
    validation_data=(X_val_small, y_val_small),
    epochs=5,
    batch_size=config['batch_size'],
    callbacks=[early_stop_quick],
    verbose=1
)

model_path = os.path.join(save_dir, f"{config['name']}_model.h5")
tuning_model.save(model_path)
print(f"\t[model saved] -> {model_path}")

history_path = os.path.join(save_dir, f"{config['name']}_history.json")
with open(history_path, "w") as f:
    json.dump(history.history, f)
print(f"\t[history saved] -> {history_path}")

# Evaluate
print("\t[model eval]")
val_loss, val_mae = tuning_model.evaluate(X_val, y_val, verbose=1)

tuning_results.append({
    'config': config['name'],
    'latent_dim': config['latent_dim'],
    'dropout': config['dropout'],
    'lr': config['lr'],
    'batch_size': config['batch_size'],
    'val_loss': val_loss,
    'val_mae': val_mae
})

print(f"    Val Loss: {val_loss:.6f} | Val MAE: {val_mae:.6f}")

```

```
>>> Config 1/3: Mas_Dropout
      latent_dim=128, dropout=0.3, lr=0.001
      [model built]
      [model init train]
Epoch 1/5
391/391 ————— 251s 615ms/step - loss: 0.0246 - mae: 0.1244 - val_loss: 0.0384 - val_mae: 0.1566
Epoch 2/5
391/391 ————— 238s 610ms/step - loss: 0.0218 - mae: 0.1183 - val_loss: 0.0396 - val_mae: 0.1589
Epoch 3/5
391/391 ————— 237s 607ms/step - loss: 0.0181 - mae: 0.1070 - val_loss: 0.0231 - val_mae: 0.1211
Epoch 4/5
391/391 ————— 231s 590ms/step - loss: 0.0165 - mae: 0.1019 - val_loss: 0.0238 - val_mae: 0.1229
Epoch 5/5
391/391 ————— 236s 602ms/step - loss: 0.0161 - mae: 0.1008 - val_loss: 0.0252 - val_mae: 0.1259
Restoring model weights from the end of the best epoch: 3.
```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

```
      [model saved] -> tuning_results\Mas_Dropout_model.h5
      [history saved] -> tuning_results\Mas_Dropout_history.json
      [model eval]
```

```
1407/1407 ————— 55s 39ms/step - loss: 0.0231 - mae: 0.1210
Val Loss: 0.023074 | Val MAE: 0.121007
```

```
>>> Config 2/3: Menos_Units
      latent_dim=64, dropout=0.2, lr=0.001
      [model built]
      [model init train]
Epoch 1/5
391/391 ————— 167s 403ms/step - loss: 0.0254 - mae: 0.1259 - val_loss: 0.0411 - val_mae: 0.1626
Epoch 2/5
391/391 ————— 149s 381ms/step - loss: 0.0218 - mae: 0.1181 - val_loss: 0.0348 - val_mae: 0.1482
Epoch 3/5
391/391 ————— 149s 381ms/step - loss: 0.0206 - mae: 0.1146 - val_loss: 0.0255 - val_mae: 0.1268
Epoch 4/5
391/391 ————— 148s 379ms/step - loss: 0.0170 - mae: 0.1038 - val_loss: 0.0245 - val_mae: 0.1243
Epoch 5/5
391/391 ————— 148s 378ms/step - loss: 0.0162 - mae: 0.1011 - val_loss: 0.0286 - val_mae: 0.1339
Restoring model weights from the end of the best epoch: 4.
```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

```

[model saved] -> tuning_results\Menos_Units_model.h5
[history saved] -> tuning_results\Menos_Units_history.json
[model eval]
1407/1407 ————— 30s 22ms/step - loss: 0.0245 - mae: 0.1243
Val Loss: 0.024468 | Val MAE: 0.124294

>>> Config 3/3: LR_Menor
      latent_dim=128, dropout=0.2, lr=0.0005
      [model built]
      [model init train]
Epoch 1/5
391/391 ————— 276s 684ms/step - loss: 0.0264 - mae: 0.1277 - val_loss: 0.0452 - val_mae: 0.1720
Epoch 2/5
391/391 ————— 268s 684ms/step - loss: 0.0221 - mae: 0.1191 - val_loss: 0.0378 - val_mae: 0.1552
Epoch 3/5
391/391 ————— 279s 714ms/step - loss: 0.0213 - mae: 0.1168 - val_loss: 0.0312 - val_mae: 0.1399
Epoch 4/5
391/391 ————— 268s 685ms/step - loss: 0.0178 - mae: 0.1062 - val_loss: 0.0249 - val_mae: 0.1253
Epoch 5/5
391/391 ————— 267s 682ms/step - loss: 0.0167 - mae: 0.1029 - val_loss: 0.0286 - val_mae: 0.1339
Restoring model weights from the end of the best epoch: 4.
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
      [model saved] -> tuning_results\LR_Menor_model.h5
      [history saved] -> tuning_results\LR_Menor_history.json
      [model eval]
1407/1407 ————— 61s 44ms/step - loss: 0.0249 - mae: 0.1252
Val Loss: 0.024912 | Val MAE: 0.125183

```

Resultados

Cambio	Validation Loss	Validation MAE
Dropout 0.2->0.3	0.0231	0.1210
Latent_dim 128->64	0.0245	0.1243
learning rate 0.001->0.0005	0.0249	0.1252

Se puede observar un rendimiento bastante comparable entre las diferentes variaciones de los modelos. El cambio que llevo a mejores resultados fue el aumento de dropout, lo cual es indicativo de una topologia que ya posee altas capacidades de acoplacion a los datos.

Asimismo, el modelo con menor lr tiene esperablemente un mayor error, pues converge mas despacio

Forecasting

Dado el modelo final generado con las arquitecturas establecidas, podemos comparar su rendimiento en el pronóstico de diferentes entradas. A continuación se evalúan predicciones para 50 artículos diferentes en 10 tiendas por el periodo de 3 meses

```
In [28]: # Cargar modelo
final_model = tf.keras.models.load_model(
    "tuning_results/Mas_Dropout_model.h5",
    custom_objects={"GRU": PatchedGRU},
    compile=False
)
final_model.compile(optimizer='adam', loss='mse', metrics=['mae'])

# Realizar predicción
y_pred_scaled = final_model.predict(X_test, verbose=1)
```

1407/1407 ————— 53s 37ms/step

```
In [29]: y_pred_original = y_pred_scaled
y_test_original = y_test

y_true_flat = y_test_original.reshape(-1)
y_pred_flat = y_pred_original.reshape(-1)
mae_final = mean_absolute_error(y_pred_flat, y_true_flat)
mse_final = mean_squared_error(y_pred_flat, y_true_flat)
rmse_final = np.sqrt(mse_final)
print(f"\nMÉTRICAS FINALES:")
print(f"    MAE: {mae_final:.4f}")
print(f"    MSE: {mse_final:.4f}")
print(f"    RMSE: {rmse_final:.4f}")
```

MÉTRICAS FINALES:

MAE: 0.1106

MSE: 0.0192

RMSE: 0.1386

```
In [28]: print(f"\n EJEMPLOS (primeras 5 secuencias):")
for i in range(min(5, len(y_pred_original))):
    real = y_test_original[i].flatten()
    pred = y_pred_original[i].flatten()

    print(f"\nSecuencia {i+1}:")
    print(f"    Real: {real}")
    print(f"    Predicción: {pred}")
```

EJEMPLOS (primeras 5 secuencias):

Secuencia 1:

Real: [97.95652174 100.69565217 87. 100.69565217 56.86956522
76.04347826 92.47826087 73.30434783 117.13043478 122.60869565
111.65217391 67.82608696 84.26086957 78.7826087 114.39130435
87. 108.91304348 100.69565217 87. 67.82608696
89.73913043 108.91304348 81.52173913 95.2173913 84.26086957
73.30434783 62.34782609 65.08695652 81.52173913 84.26086957
108.91304348 92.47826087 67.82608696 89.73913043 89.73913043
97.95652174 81.52173913 100.69565217 100.69565217 84.26086957
62.34782609 78.7826087 70.56521739 108.91304348 92.47826087
87. 65.08695652 89.73913043 84.26086957 54.13043478
95.2173913 117.13043478 84.26086957 54.13043478 70.56521739
73.30434783 84.26086957 117.13043478 84.26086957 76.04347826
45.91304348 84.26086957 78.7826087 92.47826087 89.73913043
89.73913043 95.2173913 56.86956522 67.82608696 48.65217391
84.26086957 89.73913043 87. 95.2173913 73.30434783
70.56521739 67.82608696 67.82608696 89.73913043 114.39130435
78.7826087 51.39130435 62.34782609 87. 76.04347826
89.73913043 95.2173913 70.56521739 45.91304348 62.34782609]
Predicción: [90.99168 95.48522 102.578255 108.60925 74.09924 85.6242
86.57649 91.91496 97.575836 103.781204 110.19941 75.66101
87.22245 87.990715 92.63486 99.50118 105.49549 112.65464
75.841034 87.55497 88.03114 93.80857 100.60844 103.42401
112.84434 76.380325 87.669 90.7901 95.25013 101.31548
107.354675 114.043304 76.221466 88.47149 91.259315 95.16387
101.963745 108.47906 113.54151 77.016235 88.69976 90.05067
95.836845 100.50647 107.31159 111.98078 74.339485 88.19374
87.8334 93.2873 100.00037 104.73671 109.38396 73.6037
85.91126 85.5147 91.898056 97.13321 101.023026 107.10435
69.94248 84.01396 82.85684 88.76693 94.33607 98.22354
103.85005 68.32351 81.831024 81.829834 86.50371 91.81649
96.57844 100.667595 67.61682 79.295074 80.09334 84.39024
90.598724 95.06175 98.84562 67.62593 78.937614 77.97777
84.16711 88.982124 94.28921 98.83108 69.17744 78.15414]

Secuencia 2:

Real: [100.69565217 87. 100.69565217 56.86956522 76.04347826
92.47826087 73.30434783 117.13043478 122.60869565 111.65217391
67.82608696 84.26086957 78.7826087 114.39130435 87.
108.91304348 100.69565217 87. 67.82608696 89.73913043
108.91304348 81.52173913 95.2173913 84.26086957 73.30434783
62.34782609 65.08695652 81.52173913 84.26086957 108.91304348
92.47826087 67.82608696 89.73913043 89.73913043 97.95652174
81.52173913 100.69565217 100.69565217 84.26086957 62.34782609
78.7826087 70.56521739 108.91304348 92.47826087 87.
65.08695652 89.73913043 84.26086957 54.13043478 95.2173913
117.13043478 84.26086957 54.13043478 70.56521739 73.30434783
84.26086957 117.13043478 84.26086957 76.04347826 45.91304348
84.26086957 78.7826087 92.47826087 89.73913043 89.73913043
95.2173913 56.86956522 67.82608696 48.65217391 84.26086957
89.73913043 87. 95.2173913 73.30434783 70.56521739
67.82608696 67.82608696 89.73913043 114.39130435 78.7826087
51.39130435 62.34782609 87. 76.04347826 89.73913043
95.2173913 70.56521739 45.91304348 62.34782609 54.13043478]

Predicción: [96.333145 103.59108 107.93519 74.45187 86.45954 86.9503
 93.03189 98.56188 104.1366 110.50747 76.12703 87.47143
 88.43006 93.13861 100.328766 105.93643 113.211044 75.086685
 87.87058 88.40552 94.630196 100.736984 104.44289 113.65128
 76.487335 88.09396 91.22752 95.07881 101.67038 108.03169
 114.94294 76.19961 89.05205 91.71171 95.92721 102.48538
 108.8194 114.413666 77.54824 89.42664 91.02759 96.69942
 100.82816 108.16181 113.10719 75.07409 89.44243 89.21556
 94.283516 101.12472 106.29478 111.50027 74.57638 87.35311
 86.89158 93.515884 98.68484 103.2254 109.268745 71.098785
 85.64635 84.23395 90.155846 96.09896 100.05938 106.10741
 70.70833 83.459206 82.54975 88.22766 93.2833 98.37241
 102.82713 68.34413 80.404045 80.79531 85.285576 91.38253
 96.52092 100.80133 67.89389 79.819626 78.661354 84.560905
 90.37201 95.62502 100.17355 69.15908 78.83806 77.92904]

Secuencia 3:

Real: [87. 100.69565217 56.86956522 76.04347826 92.47826087
 73.30434783 117.13043478 122.60869565 111.65217391 67.82608696
 84.26086957 78.7826087 114.39130435 87. 108.91304348
 100.69565217 87. 67.82608696 89.73913043 108.91304348
 81.52173913 95.2173913 84.26086957 73.30434783 62.34782609
 65.08695652 81.52173913 84.26086957 108.91304348 92.47826087
 67.82608696 89.73913043 89.73913043 97.95652174 81.52173913
 100.69565217 100.69565217 84.26086957 62.34782609 78.7826087
 70.56521739 108.91304348 92.47826087 87. 65.08695652
 89.73913043 84.26086957 54.13043478 95.2173913 117.13043478
 84.26086957 54.13043478 70.56521739 73.30434783 84.26086957
 117.13043478 84.26086957 76.04347826 45.91304348 84.26086957
 78.7826087 92.47826087 89.73913043 89.73913043 95.2173913
 56.86956522 67.82608696 48.65217391 84.26086957 89.73913043
 87. 95.2173913 73.30434783 70.56521739 67.82608696
 67.82608696 89.73913043 114.39130435 78.7826087 51.39130435
 62.34782609 87. 76.04347826 89.73913043 95.2173913
 70.56521739 45.91304348 62.34782609 54.13043478 67.82608696]

Predicción: [103.90855 109.31709 72.72318 87.43904 85.92419 92.91698
 98.65347 104.382225 109.698135 75.99308 88.73765 88.65849
 93.85738 101.3758 105.86052 114.04675 75.474045 88.01463
 88.82154 94.95063 101.32117 104.76818 114.056496 76.22996
 87.949684 91.14093 95.19528 101.80398 108.09718 114.8944
 76.37523 88.28415 91.22558 96.45151 102.41321 108.40012
 114.117355 76.56268 88.365906 90.70349 96.233894 100.72927
 108.24801 112.43932 74.10853 88.45566 88.84336 93.950134
 100.80563 105.63793 111.214905 73.60624 86.673584 86.61013
 93.13046 98.08983 102.99294 108.40551 70.750404 84.97119
 83.794685 89.77577 95.329765 99.823784 105.84609 70.61816
 82.87158 81.912186 87.777954 92.532524 98.14719 102.62135
 68.08752 79.96268 80.2098 84.93734 91.05323 95.94616
 100.397255 67.82574 79.22688 78.08251 84.25218 90.325935
 95.12474 99.62672 68.701675 78.49401 77.6013 84.01547]

Secuencia 4:

Real: [100.69565217 56.86956522 76.04347826 92.47826087 73.30434783
 117.13043478 122.60869565 111.65217391 67.82608696 84.26086957
 78.7826087 114.39130435 87. 108.91304348 100.69565217
 87. 67.82608696 89.73913043 108.91304348 81.52173913


```

95.2173913 84.26086957 73.30434783 62.34782609 65.08695652
81.52173913 84.26086957 108.91304348 92.47826087 67.82608696
89.73913043 89.73913043 97.95652174 81.52173913 100.69565217
100.69565217 84.26086957 62.34782609 78.7826087 70.56521739
108.91304348 92.47826087 87. 65.08695652 89.73913043
84.26086957 54.13043478 95.2173913 117.13043478 84.26086957
54.13043478 70.56521739 73.30434783 84.26086957 117.13043478
84.26086957 76.04347826 45.91304348 84.26086957 78.7826087
92.47826087 89.73913043 89.73913043 95.2173913 56.86956522
67.82608696 48.65217391 84.26086957 89.73913043 87.
95.2173913 73.30434783 70.56521739 67.82608696 67.82608696
89.73913043 114.39130435 78.7826087 51.39130435 62.34782609
87. 76.04347826 89.73913043 95.2173913 70.56521739
45.91304348 62.34782609 54.13043478 67.82608696 65.08695652]
Predicción: [105.11643 75.246414 87.08041 86.72597 91.52754 95.43471
101.9982 107.90657 73.21017 86.80833 87.32202 91.87117
99.92383 104.153625 111.88848 74.03834 86.77084 87.48333
93.50067 99.79733 104.29175 112.04163 75.57851 86.46759
89.05564 93.21538 100.515785 106.30419 111.450485 75.042015
86.48913 88.9713 93.67395 100.076416 106.15824 110.771576
75.196175 85.99492 87.53624 93.521545 98.34978 104.6507
109.002174 72.13044 85.83377 86.31092 91.18691 98.187935
102.38006 107.50748 71.721275 84.30014 84.22228 90.41981
94.90536 99.68701 104.95378 69.2923 82.78763 82.03662
87.72027 92.25699 97.38155 102.78518 68.66916 81.0633
80.09491 85.32737 89.98629 95.33958 99.5183 66.65402
78.23888 78.90024 83.21913 89.4885 93.35345 97.52198
66.94854 77.49433 76.96161 82.952644 87.95152 93.03572
97.446304 68.03162 77.31566 76.896835 82.70401 87.883675]

```

Secuencia 5:

```

Real: [ 56.86956522 76.04347826 92.47826087 73.30434783 117.13043478
122.60869565 111.65217391 67.82608696 84.26086957 78.7826087
114.39130435 87. 108.91304348 100.69565217 87.
67.82608696 89.73913043 108.91304348 81.52173913 95.2173913
84.26086957 73.30434783 62.34782609 65.08695652 81.52173913
84.26086957 108.91304348 92.47826087 67.82608696 89.73913043
89.73913043 97.95652174 81.52173913 100.69565217 100.69565217
84.26086957 62.34782609 78.7826087 70.56521739 108.91304348
92.47826087 87. 65.08695652 89.73913043 84.26086957
54.13043478 95.2173913 117.13043478 84.26086957 54.13043478
70.56521739 73.30434783 84.26086957 117.13043478 84.26086957
76.04347826 45.91304348 84.26086957 78.7826087 92.47826087
89.73913043 89.73913043 95.2173913 56.86956522 67.82608696
48.65217391 84.26086957 89.73913043 87. 95.2173913
73.30434783 70.56521739 67.82608696 67.82608696 89.73913043
114.39130435 78.7826087 51.39130435 62.34782609 87.
76.04347826 89.73913043 95.2173913 70.56521739 45.91304348
62.34782609 54.13043478 67.82608696 65.08695652 73.30434783]
Predicción: [ 78.27473 83.64717 85.98902 89.818 95.408936 101.63073
106.81948 73.712746 85.60294 86.30988 90.58311 98.24335
103.897354 111.3471 74.06559 85.90952 86.724365 92.0501
99.26186 103.85017 111.20834 75.45822 86.044365 89.2189
92.99334 100.417305 106.09152 110.26635 74.991936 86.15957
88.71866 93.49375 99.73016 105.792656 109.99024 75.28275
85.96156 87.85187 93.4646 98.12569 104.154305 108.23088

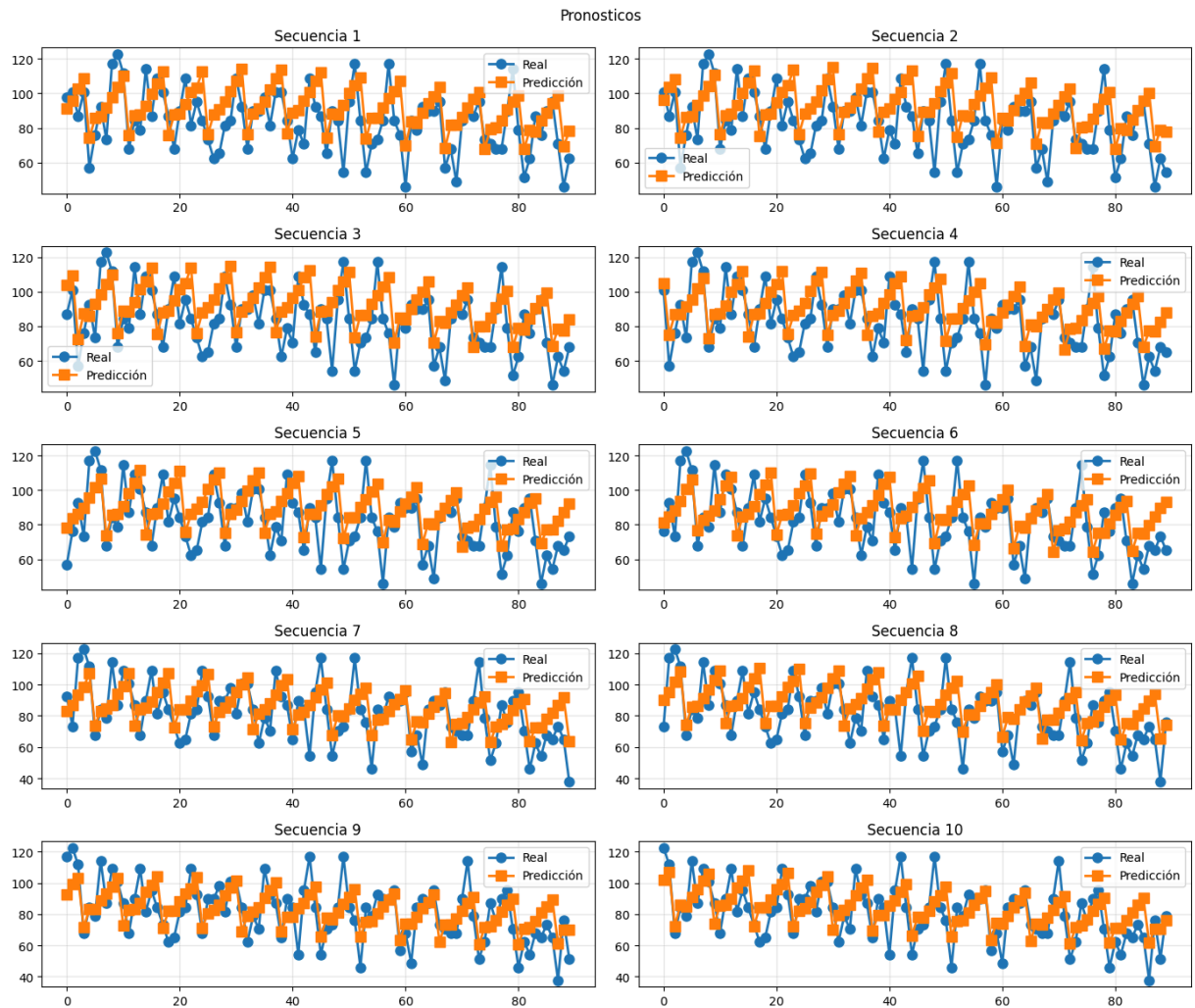
```

72.46163	86.0787	86.14187	90.989426	97.796974	102.071106
106.5345	72.09437	84.31411	84.33814	90.176834	94.54861
98.93203	103.420685	69.80789	82.784	81.84715	87.73478
92.05019	96.19204	101.84648	68.78747	80.883804	80.20518
85.136284	89.71005	94.3215	98.55086	66.97718	78.2361
79.28494	83.38307	89.087715	93.03571	96.327385	67.65428
77.12635	77.60182	83.41438	87.117935	92.625244	95.314285
69.11816	77.06474	77.25803	82.13004	87.08603	92.25403]

```
In [ ]: # Visualizacion
n_viz = min(10, len(y_pred_original))
fig, axes = plt.subplots(5, 2, figsize=(14, 12))
fig.suptitle("Pronosticos")
axes = axes.flatten()

for i in range(n_viz):
    axes[i].plot(y_test_original[i].flatten(), 'o-', label='Real', linewidth=2, mar
    axes[i].plot(y_pred_original[i].flatten(), 's-', label='Predicción', linewidth=
    axes[i].set_title(f'Secuencia {i+1}')
    axes[i].legend()
    axes[i].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('part8_forecasting.png', dpi=300, bbox_inches='tight')
plt.show()
```



Se pueden observar diferentes comportamientos interesantes en los pronosticos realizados por el modelo entrenado. Especialmente, la forma general de sierra o picos de lento aumento y rapido descenso. Esto representa la arquitectura por ventanas del modelo que tiende a alizar predicciones en un rango especifico. Utilizando lenguaje de series temporales, podemos ver que el modelo logra obviar ruido, pero captura la estacionalidad de epocas de aumento de ventas de productos y su recursivo descenso. Asimismo, muestra en todas una tendencia estacionaria o levemente negativa